PENETRATION TESTING REPORT

PixelVault CTF — Vulnerable VM Challenge

Target	PixelVault CTF Virtual Machine (.OVA)
Tester	CTF Participant
Date	2026-03-12
Engagement Type	Local Penetration Test — CTF Challenge
Classification	Confidential

1. Executive Summary

This report documents the penetration testing engagement conducted against the PixelVault CTF virtual machine. The objective was to identify security vulnerabilities, gain unauthorised access, escalate privileges to root, and retrieve both the user-level and root-level flags.

Two critical vulnerabilities were identified and successfully exploited during the engagement:

#	Vulnerability	Impact	Flag Retrieved
1	GRUB Bootloader — Unrestricted Single-User Mode Access	Root-level command execution at boot	Root Flag
2	Unsafe File Upload / Weak Web Application Access Control	Unauthenticated access to sensitive flag file	User Flag

Full root compromise of the target system was achieved. Both flags were successfully retrieved, demonstrating the complete impact of the identified vulnerabilities.

2. Methodology

The engagement followed a standard penetration testing methodology adapted for a local VM challenge environment:

Phase	Description

Reconnaissance	Identified system users and filesystem structure using /etc/passwd enumeration.
Initial Access	Exploited unsecured GRUB bootloader to gain root shell at boot time.
Credential Manipulation	Reset the ctfadmin user password using passwd command from root shell.
Privilege Escalation	Used sudo su with the newly set credentials to escalate to root.
Flag Retrieval	Located and read both flag files using find and cat commands.

3. Detailed Technical Findings

Step 1 — GRUB Bootloader Exploitation

The GRUB bootloader had no password protection. By interrupting the boot process and appending kernel parameters, it was possible to boot the system directly into a root Bash shell without any authentication.

Command entered at GRUB boot prompt:

```
rw init=/bin/bash
```

The rw parameter remounts the root filesystem in read-write mode. The init=/bin/bash parameter replaces the normal init process with a Bash shell, dropping directly into a root shell before any login mechanism is invoked.

Step 2 — User Enumeration via /etc/passwd

With root shell access, /etc/passwd was examined to enumerate system and human users.

Command:

```
cat /etc/passwd
```

This returned all system accounts. Many entries were system/service accounts with no interactive shell (nologin or false). To isolate active human users, UID filtering was applied:

```
awk -F: '$3 >= 1000 {print $1}' /etc/passwd
```

UIDs >= 1000 are conventionally assigned to regular human users on Linux systems. This revealed the active user: ctfadmin.

Step 3 — Password Reset for ctfadmin

Operating as root from the GRUB shell, the password for the ctfadmin account was reset to enable subsequent login.

Command:

```
passwd ctfadmin
# Entered: hacked
# Confirmed: hacked
```

Because the filesystem was mounted read-write (rw), the password change was persisted to /etc/shadow successfully.

Step 4 — Resume Normal Boot

After completing the password reset, the system was resumed into its normal boot sequence.

Command:

```
exec /sbin/init
```

This replaces the current Bash shell process with the normal init process, allowing the system to complete booting into a standard login environment.

Step 5 — Login as ctfadmin and Retrieve User Flag

After the system booted, login was performed as ctfadmin using the newly set password. A recursive search for flag files was then executed.

Command:

```
find / -name "flag*" -o -name "*.flag" -o -name "flag.txt" 2>/dev/null
```

This searches the entire filesystem for files matching common flag naming patterns. The 2>/dev/null suppresses permission error output. The search returned /var/www/flag.txt among other system paths.

Command:

```
cat /var/www/flag.txt
```

User Flag Retrieved:

CTF{uns4f3_upl04d_byp4ss_g0t_m3_4_sh3ll}

The flag name indicates the intended exploit path was an unsafe file upload vulnerability in the web application hosted at /var/www — confirming a second attack vector exists on this machine.

Step 6 — Privilege Escalation to Root

The ctfadmin user had sudo privileges. Privilege escalation to root was achieved using:

Command:

```
sudo su
# Password entered: hacked
```

sudo su invokes a root shell by authenticating the current user's sudo rights. The password set in Step 3 was accepted, granting a full root shell.

Step 7 — Retrieve Root Flag

As root, the flag search was repeated and the root flag was located under /root/flag.txt.

Command:

```
find / -name "flag*" -o -name "*.flag" -o -name "flag.txt" 2>/dev/null
```

Command:

```
cat /root/flag.txt
```

Root Flag Retrieved:

```
CTF{cr0n_j0b_h1j4ck_r00t3d_th3_b0x}
```

The flag name suggests the intended exploitation path to root was via a cron job hijack vulnerability — however root was obtained via the GRUB bootloader bypass in this engagement.

4. Flags Summary

Flag Type	File Path	Value
User Flag	/var/www/flag.txt	CTF{uns4f3_upl04d_byp4ss_g0t_m3_4_sh3ll}
Root Flag	/root/flag.txt	CTF{cr0n_j0b_h1j4ck_r00t3d_th3_b0x}

5. Remediation Recommendations

Finding 1 — GRUB Bootloader Has No Password Protection

An attacker with physical or console access can interrupt the boot process and gain an unauthenticated root shell by modifying kernel parameters. This completely bypasses all OS-level authentication.

Remediation:

Set a GRUB superuser password to prevent unauthorised modification of boot parameters. Generate a hashed password and add it to /etc/grub.d/40_custom:

```
# Generate hashed password
grub-mkpasswd-pbkdf2

# Add to /etc/grub.d/40_custom:
set superusers="admin"
password_pbkdf2 admin <paste-hash-here>

# Apply changes
update-grub
```

Additionally, enable Secure Boot and restrict physical access to the machine where possible.

Finding 2 — Unsafe File Upload / Unauthenticated Web Access

The flag file at /var/www/flag.txt was accessible by the ctfadmin user without requiring escalation. The flag name (uns4f3_upl04d_byp4ss) suggests the web application permitted unrestricted file uploads, enabling remote code execution or directory traversal.

Remediation:

• Restrict web-accessible directories — sensitive files such as flag.txt must never reside under the web root (/var/www).

• Implement strict file upload validation: whitelist allowed MIME types and extensions, reject executable files, and store uploads outside the web root.

• Apply least-privilege file permissions — web server processes should not have read access to sensitive system files.

• Conduct regular security audits of web application code to identify upload handler vulnerabilities.

Finding 3 — ctfadmin Has Unrestricted sudo Access

The ctfadmin user was able to run sudo su without restriction, immediately yielding a root shell. Combined with the GRUB vulnerability allowing password reset, this creates a trivial full compromise chain.

Remediation:

Apply the principle of least privilege to sudo configuration. Edit /etc/sudoers using visudo and restrict ctfadmin to only the specific commands required:

```
# /etc/sudoers — restrict ctfadmin
# BEFORE (too permissive):
ctfadmin ALL=(ALL:ALL) ALL

# AFTER (least privilege example):
ctfadmin ALL=(ALL) /usr/bin/specific-command-only
```

Avoid granting sudo access to shells (bash, sh, su) as this trivially allows privilege escalation.

Finding 4 — Bash History Disabled (Anti-Forensics)

The .bashrc for ctfadmin contained HISTSIZE=0 and unset HISTFILE, disabling bash history entirely. This is an indicator of anti-forensics activity and would hinder incident response.

Remediation:

Remove HISTSIZE=0 and unset HISTFILE from all user shell configuration files. Implement centralised audit logging (e.g. auditd) so that commands cannot be hidden by local history manipulation:

```
# Install and enable auditd
apt install auditd
systemctl enable auditd --now

# Log all executed commands
auditctl -a always,exit -F arch=b64 -S execve
```

6. Conclusion

The PixelVault CTF virtual machine was fully compromised through a chain of vulnerabilities beginning with an unprotected GRUB bootloader. This allowed unauthenticated root shell access at boot time, which was used to reset user credentials and subsequently gain full root access to the live system. Both the user flag and root flag were successfully retrieved.

The vulnerabilities identified are all well-known and entirely preventable. Implementing the remediation steps outlined in Section 5 would significantly harden the system against these attack vectors.

End of Report